A Duplication Problem

Where you and Bill face a problem with duplicated code.

Today, your boss asked you to work on a program for the accounting department. They want a system that flags expenses greater than $99 for computer gear, so they can crack down on developers splurging with company money. (You read that right: *$99*. The purchasing department isn't fooling around.)

Some other developers already took a stab at the project, coding a report that lists all the components of each computer in the company and how much each component costs. To date, they haven't plugged in any real data. Here's where you and Bill come in.

The Legacy System

Right from the start, you have a challenge on your hands: the data you need to load into the already established program is stored in a legacy system stuck behind an awkwardly coded class named `DS` (for "data source"):

methods/computer/data_source.rb

```
class DS
  def initialize # connect to data source...
  def get_cpu_info(workstation_id) # ...
  def get_cpu_price(workstation_id) # ...
  def get_mouse_info(workstation_id) # ...
  def get_mouse_price(workstation_id) # ...
  def get_keyboard_info(workstation_id) # ...
  def get_keyboard_price(workstation_id) # ...
  def get_display_info(workstation_id) # ...
  def get_display_price(workstation_id) # ...
  # ...and so on
```

`DS#initialize` connects to the data system when you create a new `DS` object. The other methods—and there are dozens of them—take a workstation identifier and return descriptions and prices for the computer's components. With Bill standing by to offer moral support, you quickly try the class in irb:

```
ds = DS.new
ds.get_cpu_info(42)     # => "2.9 Ghz quad-core"
ds.get_cpu_price(42)    # => 120
ds.get_mouse_info(42)   # => "Wireless Touch"
ds.get_mouse_price(42)  # => 60
```

It looks like workstation number 42 has a 2.9GHz CPU and a luxurious $60 mouse. This is enough data to get you started.